

Answerspot — Master Build Document

Complete developer handoff. Everything needed to create the repo and start building: architecture, decisions, and every file with its exact path.

Product: AI answer-monitoring for local service businesses. Shows operators (roofers, dentists, HVAC, auto shops) whether AI assistants recommend them, who beats them, and exactly what to fix.

Version: 0.1.0 · **Date:** 2026-07-10 · **Target infra:** Docker + Kubernetes (cloud-agnostic)

How to use this document

1. Create an empty repo.
2. Create each file at the **path shown in its heading**.
3. Fill in `.env` from `.env.example`.
4. `make dev` → `make migrate` → `make seed`.
5. Work the milestones in §14, resolving `TODO(Mx)` markers in order.

Fenced code inside this doc uses four backticks where a file itself contains triple backticks (e.g. READMEs), so copy the *inner* content.

Table of contents

1. Product & scope
2. Architecture
3. Repo layout
4. Root tooling — `package.json`, `Makefile`, `.env.example`
5. Local dev — `docker-compose.yml`
6. Data model — Prisma schema + seed
7. API — NestJS
8. Workers — Python/Celery
9. Dockerfiles
10. Helm chart
11. CI/CD — GitHub Actions
12. ADRs (decisions to resolve)
13. Handoff checklist
14. Build milestones

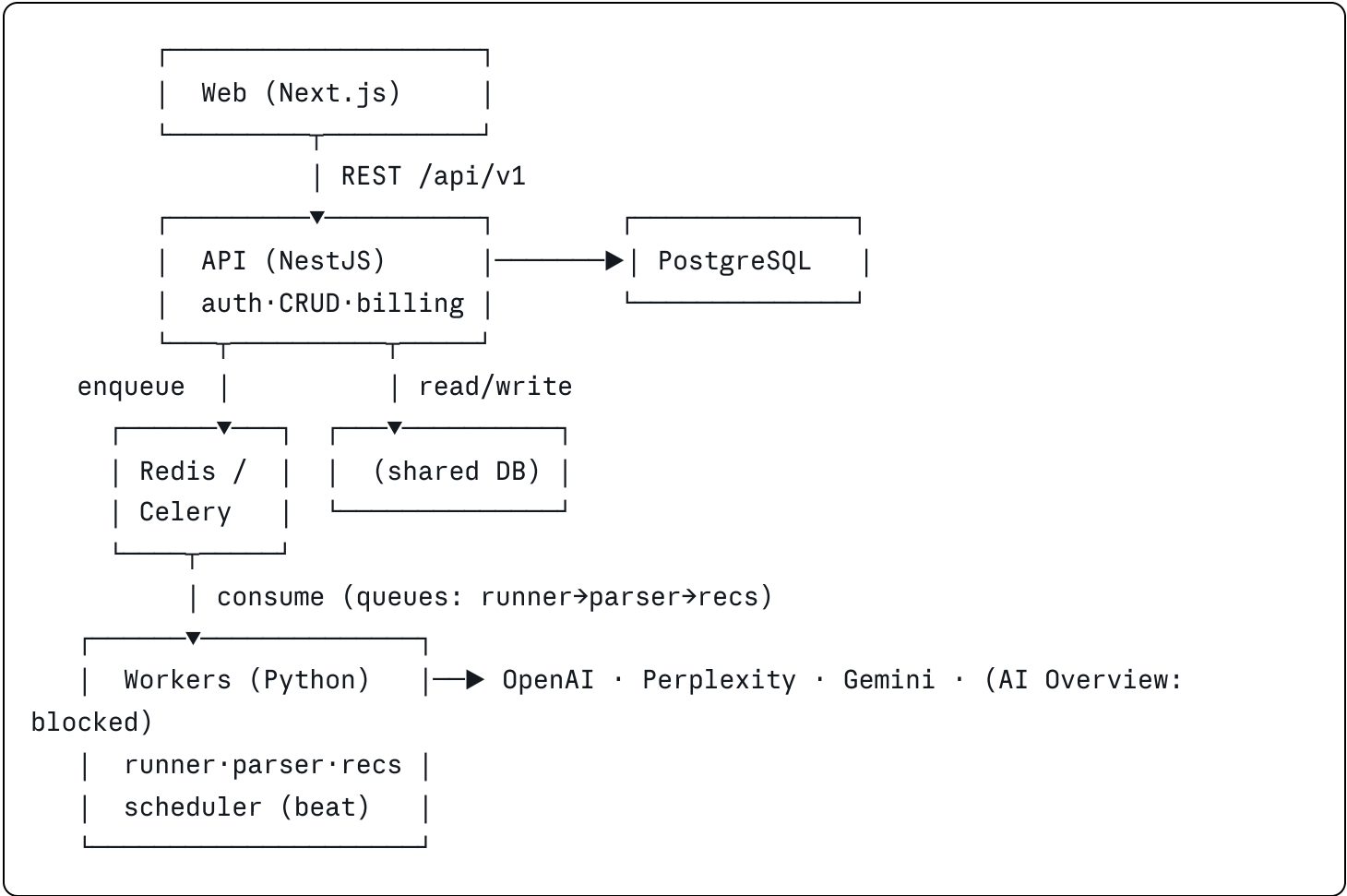
1. Product & scope

AI search replaced the 10 blue links. A homeowner asks a chat assistant "best roofer near me" and gets 3 names. Answerspot monitors what AI platforms (ChatGPT, Perplexity, Gemini, Google AI Overviews) say for the queries a business cares about, scores its visibility, ranks it against competitors, and generates plain-language fixes.

Core loop (MVP): onboard → auto-generate queries → scan platforms → show real answers + Visibility Score → deliver 3 actionable recommendations.

Non-goals v1: ad buying, generic SEO auditing, social/review generation.

2. Architecture




```
|   ├── docker/{web,api,worker}.Dockerfile
|   └── helm/
└── docs/
    ├── DESIGN_SPEC.md
    └── ADR/0001..0005.md
└── .github/workflows/{ci,deploy,release}.yaml
```

4. Root tooling

`package.json`

```
json
{
  "name": "answerspot",
  "version": "0.1.0",
  "private": true,
  "workspaces": ["apps/web", "apps/api", "packages/*"],
  "engines": { "node": ">=20" },
  "scripts": {
    "lint": "npm run lint --workspaces --if-present",
    "typecheck": "npm run typecheck --workspaces --if-present",
    "test": "npm run test --workspaces --if-present",
    "build": "npm run build --workspaces --if-present",
    "dev:web": "npm run dev --workspace=apps/web",
    "dev:api": "npm run start:dev --workspace=apps/api",
    "prisma:generate": "prisma generate --schema apps/api/prisma/schema.prisma",
    "prisma:migrate": "prisma migrate dev --schema apps/api/prisma/schema.prisma",
    "prisma:deploy": "prisma migrate deploy --schema apps/api/prisma/schema.prisma",
    "prisma:seed": "node apps/api/prisma/seed.js"
  },
  "devDependencies": {
    "prisma": "^5.18.0",
    "typescript": "^5.5.0",
    "eslint": "^9.8.0",
    "prettier": "^3.3.0"
  }
}
```

Makefile

makefile

```
.DEFAULT_GOAL := help
.PHONY: help dev down migrate seed test lint logs build clean

help: ## Show this help
    @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) \
        | awk 'BEGIN {FS = ":.*?## "}; {printf "  \033[36m%-10s\033[0m %s\n", $$1, $$2}'

dev: ## Start all services
    docker compose up --build
down: ## Stop all services
    docker compose down
migrate: ## Apply Prisma migrations
    docker compose exec api npm run prisma:deploy
seed: ## Seed reference + demo data
    docker compose exec api npm run prisma:seed
test: ## Run all tests
    npm run test
    cd apps/workers && pytest -q
lint: ## Lint + typecheck everything
    npm run lint
    npm run typecheck
    cd apps/workers && ruff check . && mypy answerspot_workers
logs: ## Tail logs
    docker compose logs -f
build: ## Build images locally
    docker compose build
clean: ## Remove volumes (DESTRUCTIVE)
    docker compose down -v
```

.env.example

bash

```
# Copy to .env. NEVER commit .env. Prod uses K8s Secrets/Vault.
NODE_ENV=development
LOG_LEVEL=info

DATABASE_URL=postgres://answerspot:answerspot@localhost:5432/answerspot
REDIS_URL=redis://localhost:6379
CELERY_BROKER_URL=redis://localhost:6379/1
CELERY_RESULT_BACKEND=redis://localhost:6379/2

JWT_SECRET=change-me-to-a-long-random-string
JWT_EXPIRES_IN=7d

S3_ENDPOINT=http://localhost:9000
S3_BUCKET=answerspot
S3_KEY=answerspot
S3_SECRET=answerspot123
S3_REGION=us-east-1

STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
STRIPE_PRICE_STARTER=price_...
STRIPE_PRICE_PRO=price_...
STRIPE_PRICE_AGENCY=price_...

OPENAI_API_KEY=sk-...
PERPLEXITY_API_KEY=pplx-...
GEMINI_API_KEY=...
SERP_API_KEY=

SCAN_SAMPLE_COUNT=3
SCAN_MONTHLY_BUDGET_USD=500

NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1
SENTRY_DSN=
OTEL_EXPORTER_OTLP_ENDPOINT=
```

5. Local dev — `docker-compose.yml`

```
yaml
```

```
version: "3.9"
name: answerspot

services:
  web:
    build: { context: ., dockerfile: infra/docker/web.Dockerfile }
    ports: ["3000:3000"]
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:4000/api/v1
    depends_on: [api]
    command: npm run dev --workspace=apps/web

  api:
    build: { context: ., dockerfile: infra/docker/api.Dockerfile }
    ports: ["4000:4000"]
    env_file: .env
    environment:
      DATABASE_URL: postgres://answerspot:answerspot@postgres:5432/answerspot
      REDIS_URL: redis://redis:6379
      S3_ENDPOINT: http://minio:9000
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_started }
      minio: { condition: service_started }
    command: npm run start:dev --workspace=apps/api

  worker:
    build: { context: ., dockerfile: infra/docker/worker.Dockerfile }
    env_file: .env
    environment:
      DATABASE_URL: postgres://answerspot:answerspot@postgres:5432/answerspot
      CELERY_BROKER_URL: redis://redis:6379/1
      CELERY_RESULT_BACKEND: redis://redis:6379/2
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_started }
    command: celery -A answerspot_workers worker --loglevel=info --concurrency=4

  scheduler:
    build: { context: ., dockerfile: infra/docker/worker.Dockerfile }
    env_file: .env
    environment:
      DATABASE_URL: postgres://answerspot:answerspot@postgres:5432/answerspot
```

```
    CELERY_BROKER_URL: redis://redis:6379/1
depends_on: [redis]
command: celery -A answerspot_workers beat --loglevel=info
```

postgres:

```
image: postgres:16-alpine
ports: ["5432:5432"]
environment:
    POSTGRES_USER: answerspot
    POSTGRES_PASSWORD: answerspot
    POSTGRES_DB: answerspot
volumes: [pgdata:/var/lib/postgresql/data]
healthcheck:
    test: ["CMD-SHELL", "pg_isready -U answerspot"]
    interval: 5s
    timeout: 5s
    retries: 5
```

redis:

```
image: redis:7-alpine
ports: ["6379:6379"]
volumes: [redisdata:/data]
```

minio:

```
image: minio/minio:latest
ports: ["9000:9000", "9001:9001"]
environment:
    MINIO_ROOT_USER: answerspot
    MINIO_ROOT_PASSWORD: answerspot123
command: server /data --console-address ":9001"
volumes: [miniodata:/data]
```

volumes:

```
pgdata:
redisdata:
miniodata:
```


6. Data model

apps/api/prisma/schema.prisma

prisma

```

generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql"; url = env("DATABASE_URL") }

enum Plan { STARTER PRO AGENCY }
enum OrgStatus { ACTIVE PAST_DUE CANCELED TRIALING }
enum Role { OWNER ADMIN MEMBER }
enum PlatformKey { CHATGPT PERPLEXITY GEMINI AI_OVERVIEW }
enum ScanStatus { QUEUED RUNNING DONE FAILED }
enum Sentiment { POSITIVE NEUTRAL NEGATIVE UNKNOWN }
enum RecommendationType { KEYWORD_GAP SCHEMA_MARKUP REVIEW_SIGNAL CITATION_GAP SENTIMENT }
enum Severity { LOW MEDIUM HIGH CRITICAL }
enum RecommendationStatus { OPEN RESOLVED DISMISSED }
enum AlertType { SCORE_DROP COMPETITOR_OVERTAKE SENTIMENT_NEGATIVE SCAN_FAILED }
enum AlertChannel { EMAIL SMS }
enum SubscriptionStatus { ACTIVE PAST_DUE CANCELED TRIALING INCOMPLETE }

model Organization {
  id String @id @default(cuid())
  name String
  plan Plan @default(STARTER)
  status OrgStatus @default(TRIALING)
  stripeCustomerId String? @unique
  users User[]
  memberships Membership[]
  businesses Business[]
  subscriptions Subscription[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  @@index([status])
  @@map("organizations")
}

model User {
  id String @id @default(cuid())
  organizationId String
  email String @unique
  name String?
  role Role @default(MEMBER)
  authProviderId String? @unique
  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
  memberships Membership[]
  createdAt DateTime @default(now())
}

```

```

        updatedAt DateTime @updatedAt
        @@index([organizationId])
        @@map("users")
    }

model Membership {
    id String @id @default(cuid())
    userId String
    organizationId String
    role Role @default(MEMBER)
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
    createdAt DateTime @default(now())
    @@unique([userId, organizationId])
    @@index([organizationId])
    @@map("memberships")
}

model Business {
    id String @id @default(cuid())
    organizationId String
    name String
    category String
    city String
    state String
    website String?
    gbpId String?
    organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
    trackedQueries TrackedQuery[]
    visibilityScores VisibilityScore[]
    recommendations Recommendation[]
    alerts Alert[]
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
    deletedAt DateTime?
    @@index([organizationId])
    @@index([category, city, state])
    @@map("businesses")
}

model Platform {
    id String @id @default(cuid())
    key PlatformKey @unique
    displayName String

```

```

    enabled Boolean @default(true)
    scans Scan[]
    @@map("platforms")
}

model TrackedQuery {
    id String @id @default(cuid())
    businessId String
    queryText String
    location String?
    intent String?
    isActive Boolean @default(true)
    business Business @relation(fields: [businessId], references: [id], onDelete: Cascade)
    scans Scan[]
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
    @@index([businessId, isActive])
    @@map("tracked_queries")
}

model Scan {
    id String @id @default(cuid())
    trackedQueryId String
    platformId String
    status ScanStatus @default(QUEUED)
    samples Int @default(3)
    runAt DateTime?
    error String?
    trackedQuery TrackedQuery @relation(fields: [trackedQueryId], references: [id], onDelete: Cascade)
    platform Platform @relation(fields: [platformId], references: [id])
    results ScanResult[]
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
    @@index([trackedQueryId, platformId])
    @@index([status])
    @@map("scans")
}

model ScanResult {
    id String @id @default(cuid())
    scanId String
    rawResponse String @db.Text
    mentioned Boolean @default(false)
    rankPosition Int?

```

```

    sentiment Sentiment @default(UNKNOWN)
    confidence Decimal @default(0) @db.Decimal(4, 3)
    citations Json @default("[]")
    competitors Json @default("[]")
    scan Scan @relation(fields: [scanId], references: [id], onDelete: Cascade)
    createdAt DateTime @default(now())
    @@index([scanId])
    @@index([mentioned])
    @@map("scan_results")
}

model VisibilityScore {
    id String @id @default(cuid())
    businessId String
    score Int
    breakdown Json
    computedAt DateTime @default(now())
    business Business @relation(fields: [businessId], references: [id], onDelete: Cascade)
    @@index([businessId, computedAt])
    @@map("visibility_scores")
}

model Recommendation {
    id String @id @default(cuid())
    businessId String
    type RecommendationType
    severity Severity @default(MEDIUM)
    title String
    message String @db.Text
    artifact Json?
    status RecommendationStatus @default(OPEN)
    business Business @relation(fields: [businessId], references: [id], onDelete: Cascade)
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
    @@index([businessId, status])
    @@map("recommendations")
}

model Alert {
    id String @id @default(cuid())
    businessId String
    type AlertType
    channel AlertChannel
    payload Json

```

```

    sentAt DateTime?
    business Business @relation(fields: [businessId], references: [id], onDelete: Cascade)
    createdAt DateTime @default(now())
    @@index([businessId, type])
    @@map("alerts")
  }

model Subscription {
  id String @id @default(cuid())
  organizationId String
  stripeSubscriptionId String @unique
  plan Plan
  status SubscriptionStatus @default(INCOMPLETE)
  currentPeriodEnd DateTime?
  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([organizationId])
  @@map("subscriptions")
}

```

apps/api/prisma/seed.js

javascript

```

const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

async function main() {
  const platforms = [
    { key: 'CHATGPT', displayName: 'ChatGPT' },
    { key: 'PERPLEXITY', displayName: 'Perplexity' },
    { key: 'GEMINI', displayName: 'Gemini' },
    { key: 'AI_OVERVIEW', displayName: 'Google AI Overview', enabled: false },
  ];
  for (const p of platforms) {
    await prisma.platform.upsert({
      where: { key: p.key },
      update: { displayName: p.displayName, enabled: p.enabled ?? true },
      create: p,
    });
  }
  const org = await prisma.organization.upsert({
    where: { stripeCustomerId: 'demo' },
    update: {},
    create: { name: 'Demo Roofing Co', plan: 'PRO', status: 'TRIALING', stripeCustomerId: 'demo' },
  });
  await prisma.business.create({
    data: { organizationId: org.id, name: 'Demo Roofing Co', category: 'roofer',
      city: 'Austin', state: 'TX', website: 'https://example.com' },
  });
  console.log('Seed complete.');
```

```

}
main().catch((e) => { console.error(e); process.exit(1); }).finally(() => prisma.$disconnect());

```

7. API — NestJS

apps/api/src/main.ts

typescript

```

import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule, { rawBody: true });
  app.setGlobalPrefix('api/v1');
  app.enableCors({ origin: process.env.WEB_ORIGIN?.split(',') ?? true });
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true, forbidNonConformant: true }));
  const config = new DocumentBuilder().setTitle('Answerspot API').setVersion('1.0').addSecurity('bearerAuth', { type: 'bearer' });
  SwaggerModule.setup('docs', app, SwaggerModule.createDocument(app, config));
  await app.listen(process.env.PORT ?? 4000);
}
bootstrap();

```

apps/api/src/app.module.ts

typescript

```

import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { PrismaService } from '../prisma/prisma.service';
import { HealthModule } from '../health/health.module';
import { AuthModule } from '../auth/auth.module';
import { BusinessesModule } from '../businesses/businesses.module';
import { QueriesModule } from '../queries/queries.module';
import { ScansModule } from '../scans/scans.module';
import { ReportsModule } from '../reports/reports.module';
import { BillingModule } from '../billing/billing.module';

@Module({
  imports: [ConfigModule.forRoot({ isGlobal: true }), HealthModule, AuthModule,
    BusinessesModule, QueriesModule, ScansModule, ReportsModule, BillingModule],
  providers: [PrismaService],
  exports: [PrismaService],
})
export class AppModule {}

```


apps/api/src/prisma/prisma.service.ts

typescript

```
import { Injectable, OnModuleInit } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit {
  async onModuleInit() { await this.$connect(); }
}
```

apps/api/src/health/health.controller.ts

typescript

```
import { Controller, Get } from '@nestjs/common';
import { ApiTags } from '@nestjs/swagger';
import { PrismaService } from '../prisma/prisma.service';

@ApiTags('health')
@Controller('health')
export class HealthController {
  constructor(private prisma: PrismaService) {}
  @Get() liveness() { return { status: 'ok', ts: new Date().toISOString() }; }
  @Get('ready') async readiness() {
    try { await this.prisma.$queryRaw`SELECT 1`; return { status: 'ready', db: 'up' } }
    catch { return { status: 'not-ready', db: 'down' }; }
  }
}
```

apps/api/src/health/health.module.ts

typescript

```
import { Module } from '@nestjs/common';
import { HealthController } from './health.controller';
import { PrismaService } from '../prisma/prisma.service';

@Module({ controllers: [HealthController], providers: [PrismaService] })
export class HealthModule {}
```

apps/api/src/common/decorators/current-user.decorator.ts

typescript

```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';

export interface AuthUser {
  id: string; organizationId: string; role: 'OWNER' | 'ADMIN' | 'MEMBER'; email: string;
}

export const CurrentUser = createParamDecorator(
  (_d: unknown, ctx: ExecutionContext): AuthUser => ctx.switchToHttp().getRequest().user
);
```

apps/api/src/common/decorators/roles.decorator.ts

typescript

```
import { SetMetadata } from '@nestjs/common';

export const ROLES_KEY = 'roles';

export const Roles = (...roles: Array<'OWNER' | 'ADMIN' | 'MEMBER'>) => SetMetadata(ROLES_KEY, roles);
```

apps/api/src/common/guards/roles.guard.ts

typescript

```
import { CanActivate, ExecutionContext, ForbiddenException, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { ROLES_KEY } from '../decorators/roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(ctx: ExecutionContext): boolean {
    const required = this.reflector.getAllAndOverride<string[]>(ROLES_KEY, [ctx.getHandler()]);
    if (!required?.length) return true;
    const { user } = ctx.switchToHttp().getRequest();
    if (!user || !required.includes(user.role)) throw new ForbiddenException('Insufficient permissions');
    return true;
  }
}
```

apps/api/src/common/guards/org-scope.guard.ts

typescript

```
import { CanActivate, ExecutionContext, ForbiddenException, Injectable } from '@nestjs/common';
import { PrismaService } from '../../../prisma/prisma.service';

@Injectable()
export class OrgScopeGuard implements CanActivate {
  constructor(private prisma: PrismaService) {}
  async canActivate(ctx: ExecutionContext): Promise<boolean> {
    const req = ctx.switchToHttp().getRequest();
    const user = req.user;
    if (!user?.organizationId) throw new ForbiddenException('No organization context');
    const businessId = req.params.businessId ?? req.params.id;
    if (!businessId) return true;
    const business = await this.prisma.business.findFirst({
      where: { id: businessId, organizationId: user.organizationId, deletedAt: null },
      select: { id: true },
    });
    if (!business) throw new ForbiddenException('Resource not in your organization');
    req.business = business;
    return true;
  }
}
```

apps/api/src/common/dto/pagination.dto.ts

typescript

```
import { IsInt, IsOptional, Max, Min } from 'class-validator';
import { Type } from 'class-transformer';

export class PaginationDto {
  @IsOptional() @Type(() => Number) @IsInt() @Min(1) page = 1;
  @IsOptional() @Type(() => Number) @IsInt() @Min(1) @Max(100) limit = 20;
}
```

apps/api/src/businesses/dto/create-business.dto.ts

typescript

```
import { IsOptional, IsString, MaxLength } from 'class-validator';

export class CreateBusinessDto {
  @IsString() @MaxLength(200) name!: string;
  @IsString() @MaxLength(100) category!: string;
  @IsString() @MaxLength(100) city!: string;
  @IsString() @MaxLength(50) state!: string;
  @IsOptional() @IsString() website?: string;
}
```

apps/api/src/businesses/businesses.controller.ts

typescript

```
import { Body, Controller, Get, Param, Post, Query, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { BusinessesService } from '../businesses.service';
import { CreateBusinessDto } from '../dto/create-business.dto';
import { OrgScopeGuard } from '../common/guards/org-scope.guard';
import { CurrentUser, AuthUser } from '../common/decorators/current-user.decorator';
import { PaginationDto } from '../common/dto/pagination.dto';

@ApiTags('businesses')
@ApiBearerAuth()
@UseGuards(AuthGuard('jwt'), OrgScopeGuard)
@Controller('businesses')
export class BusinessesController {
  constructor(private readonly svc: BusinessesService) {}

  @Post() create(@CurrentUser() u: AuthUser, @Body() dto: CreateBusinessDto) { return }
  @Get() list(@CurrentUser() u: AuthUser, @Query() page: PaginationDto) { return this }
  @Get(':id') get(@Param('id') id: string) { return this.svc.get(id); }
  @Get(':id/visibility-score') visibilityScore(@Param('id') id: string) { return this }
  @Get(':id/competitors') competitors(@Param('id') id: string) { return this.svc.comp
  @Get(':id/recommendations') recommendations(@Param('id') id: string) { return this.
}
```

apps/api/src/businesses/businesses.service.ts

typescript

```

import { Injectable, NotFoundException } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';
import { CreateBusinessDto } from '../dto/create-business.dto';
import { PaginationDto } from '../common/dto/pagination.dto';

@Injectable()
export class BusinessesService {
  constructor(private prisma: PrismaService) {}

  async create(organizationId: string, dto: CreateBusinessDto) {
    const business = await this.prisma.business.create({ data: { ...dto, organizationId },
    // TODO(M1): enqueue query auto-generation + onboarding scan.
    });
    return business;
  }

  async list(organizationId: string, { page, limit }: PaginationDto) {
    const [data, total] = await Promise.all([
      this.prisma.business.findMany({ where: { organizationId, deletedAt: null },
        skip: (page - 1) * limit, take: limit, orderBy: { createdAt: 'desc' } }),
      this.prisma.business.count({ where: { organizationId, deletedAt: null } })
    ]);
    return { data, total, page, limit };
  }

  async get(id: string) {
    const b = await this.prisma.business.findUnique({ where: { id } });
    if (!b) throw new NotFoundException();
    return b;
  }

  async visibilityScore(businessId: string) {
    const [current, trend] = await Promise.all([
      this.prisma.visibilityScore.findFirst({ where: { businessId }, orderBy: { computedAt: 'desc' } }),
      this.prisma.visibilityScore.findMany({ where: { businessId }, orderBy: { computedAt: 'desc' },
        take: 90, select: { score: true, computedAt: true } })
    ]);
    return { current, trend };
  }

  competitors(businessId: string) {
    // TODO(M5): aggregate competitors JSONB across recent scan_results.
    return this.prisma.scanResult.findMany({
      where: { scan: { trackedQuery: { businessId } } },
      select: { competitors: true, createdAt: true },
      orderBy: { createdAt: 'desc' }, take: 50,
    });
  }
}

```

```

recommendations(businessId: string) {
  return this.prisma.recommendation.findMany({
    where: { businessId, status: 'OPEN' },
    orderBy: [{ severity: 'desc' }, { createdAt: 'desc' }],
  });
}
}

```

apps/api/src/queries/queries.controller.ts

typescript

```

import { Body, Controller, Get, Param, Patch, Post, Query, UseGuards } from '@nestjs/
import { AuthGuard } from '@nestjs/passport';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { QueriesService } from '../queries.service';
import { OrgScopeGuard } from '../../common/guards/org-scope.guard';

@ApiTags('queries')
@ApiBearerAuth()
@UseGuards(AuthGuard('jwt'), OrgScopeGuard)
@Controller()
export class QueriesController {
  constructor(private readonly svc: QueriesService) {}

  @Post('businesses/:businessId/queries')
  create(@Param('businessId') businessId: string, @Body() body: { queryText: string;
    return this.svc.create(businessId, body);
  }

  @Get('businesses/:businessId/queries')
  list(@Param('businessId') businessId: string) { return this.svc.list(businessId); }

  @Patch('queries/:id')
  update(@Param('id') id: string, @Body() body: { isActive?: boolean; queryText?: str
    return this.svc.update(id, body);
  }

  @Get('queries/:id/results')
  results(@Param('id') id: string, @Query('platform') platform?: string, @Query('rang
    return this.svc.results(id, { platform, range });
  }
}

```

apps/api/src/scans/scans.controller.ts

typescript

```
import { Body, Controller, Get, Param, Post, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { ScansService } from '../scans.service';

@ApiTags('scans')
@ApiBearerAuth()
@UseGuards(AuthGuard('jwt'))
@Controller('scans')
export class ScansController {
  constructor(private readonly svc: ScansService) {}
  @Post('trigger') trigger(@Body() body: { trackedQueryId: string; platformKeys?: string[] }) {
    return this.svc.trigger(body);
  }
  @Get('/:id') get(@Param('id') id: string) { return this.svc.get(id); }
}
```

apps/api/src/scans/scans.service.ts

typescript

```
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';

@Injectable()
export class ScansService {
  constructor(private prisma: PrismaService) {}
  async trigger(body: { trackedQueryId: string; platformKeys?: string[] }) {
    // TODO(M2): rate-limit, create Scan rows (QUEUED), enqueue scan.run.
    return { queued: true, trackedQueryId: body.trackedQueryId };
  }
  get(id: string) { return this.prisma.scan.findUnique({ where: { id }, include: { re
  }
}
```

apps/api/src/scans/scans.module.ts (pattern for all feature modules)

typescript

```
import { Module } from '@nestjs/common';
import { ScansController } from '../scans.controller';
import { ScansService } from '../scans.service';
import { PrismaService } from '../prisma/prisma.service';

@Module({ controllers: [ScansController], providers: [ScansService, PrismaService] })
export class ScansModule {}
```

apps/api/src/billing/billing.controller.ts

typescript

```
import { Controller, Headers, Post, RawBodyRequest, Req } from '@nestjs/common';
import { Request } from 'express';
import { BillingService } from '../billing.service';

@Controller()
export class BillingController {
  constructor(private readonly svc: BillingService) {}
  @Post('webhooks/stripe')
  webhook(@Req() req: RawBodyRequest<Request>, @Headers('stripe-signature') sig: stri
    return this.svc.handleWebhook(req.rawBody!, sig);
  }
}
```

API surface (v1): `POST /auth/session`, `GET /me`, businesses CRUD + `/visibility-score`
`/competitors` `/recommendations`, queries CRUD + `/results`, `POST /scans/trigger`, `GET`
`/scans/:id`, `GET /reports/:businessId`, `POST /webhooks/stripe`, `GET /health`. Apply the
module/service pattern above to `auth`, `queries`, `reports`, `billing`.

8. Workers — Python/Celery

apps/workers/pyproject.toml

toml


```
[
project
]
name = "answerspot-workers"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "celery[redis]>=5.4", "sqlalchemy>=2.0", "psycopg[binary]>=3.2",
    "pydantic>=2.7", "pydantic-settings>=2.3", "httpx>=0.27", "sentry-sdk>=2.0",
]
[
project.optional-dependencies
]
dev = ["pytest>=8", "pytest-asyncio>=0.23", "ruff>=0.5", "mypy>=1.10"]
```

`apps/workers/answerspot_workers/__init__.py`

python

```
from celery import Celery
from .config import settings

app = Celery(
    "answerspot_workers",
    broker=settings.celery_broker_url,
    backend=settings.celery_result_backend,
    include=[
        "answerspot_workers.tasks.runner",
        "answerspot_workers.tasks.parser",
        "answerspot_workers.tasks.recommender",
        "answerspot_workers.tasks.scheduler",
    ],
)
app.conf.task_routes = {
    "answerspot_workers.tasks.runner.*": {"queue": "runner"},
    "answerspot_workers.tasks.parser.*": {"queue": "parser"},
    "answerspot_workers.tasks.recommender.*": {"queue": "recs"},
}
app.conf.beat_schedule = {
    "enqueue-due-scans": {
        "task": "answerspot_workers.tasks.scheduler.enqueue_due_scans",
        "schedule": 3600.0,
    },
}
app.conf.task_acks_late = True
app.conf.task_reject_on_worker_lost = True
```

apps/workers/answerspot_workers/config.py

python

```

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    database_url: str
    celery_broker_url: str
    celery_result_backend: str
    openai_api_key: str = ""
    perplexity_api_key: str = ""
    gemini_api_key: str = ""
    serp_api_key: str = ""
    scan_sample_count: int = 3
    scan_monthly_budget_usd: float = 500.0
    sentry_dsn: str = ""

settings = Settings()

```

apps/workers/answerspot_workers/db.py

```

python

from contextlib import contextmanager
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from .config import settings

engine = create_engine(settings.database_url, pool_pre_ping=True, future=True)
SessionLocal = sessionmaker(bind=engine, expire_on_commit=False, future=True)

@contextmanager
def session_scope():
    session = SessionLocal()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()

```

apps/workers/answerspot_workers/schemas.py

python

```
from enum import Enum
from typing import Optional
from pydantic import BaseModel, Field

class Sentiment(str, Enum):
    POSITIVE = "POSITIVE"; NEUTRAL = "NEUTRAL"; NEGATIVE = "NEGATIVE"; UNKNOWN = "UNKNOWN"

class Citation(BaseModel):
    url: str; title: Optional[str] = None; source: Optional[str] = None

class CompetitorMention(BaseModel):
    name: str; rank_position: Optional[int] = None

class RawAnswer(BaseModel):
    platform_key: str; query_text: str; text: str; latency_ms: int; cost_usd: float = 0.0

class ParsedResult(BaseModel):
    mentioned: bool
    rank_position: Optional[int] = None
    sentiment: Sentiment = Sentiment.UNKNOWN
    confidence: float = Field(ge=0.0, le=1.0, default=0.0)
    citations: list[Citation] = []
    competitors: list[CompetitorMention] = []
```

apps/workers/answerspot_workers/adapters/base.py

python

```
from abc import ABC, abstractmethod
from ..schemas import RawAnswer

class PlatformAdapter(ABC):
    key: str
    @abstractmethod
    def query(self, query_text: str, location: str | None = None) -> RawAnswer: ...
```

apps/workers/answerspot_workers/adapters/perplexity_adapter.py

python

```

import time
from .base import PlatformAdapter
from ..schemas import RawAnswer
from ..config import settings

class PerplexityAdapter(PlatformAdapter):
    key = "PERPLEXITY"
    def query(self, query_text: str, location: str | None = None) -> RawAnswer:
        prompt = query_text if not location else f"{query_text} in {location}"
        started = time.monotonic()
        # TODO(M2): call Perplexity API with settings.perplexity_api_key.
        text = ""; cost = 0.0
        return RawAnswer(platform_key=self.key, query_text=prompt, text=text,
                        latency_ms=int((time.monotonic() - started) * 1000), cost_us

```

apps/workers/answerspot_workers/adapters/ai_overview_adapter.py

python

```

from .base import PlatformAdapter
from ..schemas import RawAnswer

class AIOverviewAdapter(PlatformAdapter):
    key = "AI_OVERVIEW"
    def query(self, query_text: str, location: str | None = None) -> RawAnswer:
        # BLOCKED pending legal review – see ADR 0004. Do NOT implement M0-M4.
        raise NotImplementedError("AI Overview capture blocked pending ADR 0004")

```

apps/workers/answerspot_workers/adapters/registry.py

python

```

from .base import PlatformAdapter
from .perplexity_adapter import PerplexityAdapter

_ADAPTERS: dict[str, PlatformAdapter] = { a.key: a for a in [PerplexityAdapter()] }

def get_adapter(platform_key: str) -> PlatformAdapter:
    try:
        return _ADAPTERS[platform_key]
    except KeyError:
        raise ValueError(f"No adapter for platform '{platform_key}'")

```

apps/workers/answerspot_workers/tasks/runner.py

```

python

from celery import shared_task
from celery.utils.log import get_task_logger
from ..adapters.registry import get_adapter
from ..config import settings
from ..db import session_scope
from .parser import parse_scan

log = get_task_logger(__name__)

@shared_task(name="answerspot_workers.tasks.runner.run_scan",
             bind=True, max_retries=3, default_retry_delay=30, acks_late=True)
def run_scan(self, scan_id: str, platform_key: str, query_text: str, location: str |
             adapter = get_adapter(platform_key)
    try:
        answers = [adapter.query(query_text, location) for _ in range(settings.scan_s
    except Exception as exc:
        log.exception("scan.run failed scan=%s platform=%s", scan_id, platform_key)
        raise self.retry(exc=exc)
    with session_scope() as db:
        # TODO(M2): update Scan status; insert raw sample rows + cost.
        _ = db
    parse_scan.delay(scan_id=scan_id)
    return {"scan_id": scan_id, "samples": len(answers)}

```

`apps/workers/answerspot_workers/tasks/parser.py`

python

```
from celery import shared_task
from celery.utils.log import get_task_logger
from pydantic import ValidationError
from ..schemas import ParsedResult
from ..db import session_scope
from .recommender import generate_recommendations

log = get_task_logger(__name__)

@shared_task(name="answerspot_workers.tasks.parser.parse_scan",
             bind=True, max_retries=2, default_retry_delay=15, acks_late=True)
def parse_scan(self, scan_id: str):
    with session_scope() as db:
        raw_samples: list[str] = [] # TODO(M3): load raw samples
        _ = db
        parsed: list[ParsedResult] = []
        for raw in raw_samples:
            try:
                parsed.append(ParsedResult.model_validate_json(raw)) # TODO(M3): LLM ext
            except ValidationError as exc:
                log.warning("parse invalid JSON scan=%s: %s", scan_id, exc)
                raise self.retry(exc=exc)
        with session_scope() as db:
            # TODO(M3): aggregate parsed[] → scan_result + confidence.
            _ = db
        generate_recommendations.delay(scan_id=scan_id)
        return {"scan_id": scan_id, "parsed": len(parsed)}
```

`apps/workers/answerspot_workers/tasks/recommender.py`

python

```

from celery import shared_task
from ..db import session_scope

@shared_task(name="answerspot_workers.tasks.recommender.generate_recommendations", auto_retry=True)
def generate_recommendations(scan_id: str):
    with session_scope() as db:
        # TODO(M5): rules + artifacts; (re)compute VisibilityScore snapshot
        # storing weights in breakdown (ADR 0005).
        _ = db
    return {"scan_id": scan_id, "recommendations": 0}

```

apps/workers/answerspot_workers/tasks/scheduler.py

```

python

from celery import shared_task
from ..db import session_scope
from .runner import run_scan

CADENCE_HOURS = {"STARTER": 24 * 7, "PRO": 24, "AGENCY": 24}

@shared_task(name="answerspot_workers.tasks.scheduler.enqueue_due_scans")
def enqueue_due_scans():
    enqueued = 0
    with session_scope() as db:
        due: list[dict] = [] # TODO(M2): find due (business × query × platform)
        _ = db
    for item in due:
        run_scan.delay(scan_id=item["scan_id"], platform_key=item["platform_key"],
                       query_text=item["query_text"], location=item.get("location"))
        enqueued += 1
    return {"enqueued": enqueued}

```

9. Dockerfiles

infra/docker/api.Dockerfile

dockerfile


```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
COPY apps/api/package.json apps/api/
RUN npm ci

FROM node:20-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate --schema apps/api/prisma/schema.prisma
RUN npm run build --workspace=apps/api

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -g 1001 nodejs && adduser -u 1001 -G nodejs -S nestjs
COPY --from=build --chown=nestjs:nodejs /app/node_modules ./node_modules
COPY --from=build --chown=nestjs:nodejs /app/apps/api/dist ./apps/api/dist
COPY --from=build --chown=nestjs:nodejs /app/apps/api/prisma ./apps/api/prisma
USER nestjs
EXPOSE 4000
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -qO- http://localhost:4000/api/v1/health || exit 1
CMD ["node", "apps/api/dist/main.js"]
```

infra/docker/web.Dockerfile

dockerfile

```

FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
COPY apps/web/package.json apps/web/
RUN npm ci

FROM node:20-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build --workspace=apps/web

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -g 1001 nodejs && adduser -u 1001 -G nodejs -S nextjs
COPY --from=build --chown=nextjs:nodejs /app/apps/web/.next/standalone ./
COPY --from=build --chown=nextjs:nodejs /app/apps/web/.next/static ./apps/web/.next/static
COPY --from=build --chown=nextjs:nodejs /app/apps/web/public ./apps/web/public
USER nextjs
EXPOSE 3000
CMD ["node", "apps/web/server.js"]

```

infra/docker/worker.Dockerfile

```

dockerfile

FROM python:3.12-slim AS base
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends build-essential libpq-dev
&& rm -rf /var/lib/apt/lists/*
COPY apps/workers/pyproject.toml ./
RUN pip install --upgrade pip && pip install -e .
COPY apps/workers/ ./
RUN useradd -m -u 1001 celery
USER celery
CMD ["celery", "-A", "answerspot_workers", "worker", "-Q", "runner", "--loglevel=info"]

```

10. Helm chart

`Chart.yaml`, `values.yaml` (+ `values.staging.yaml`, `values.prod.yaml`), and `templates/` for web, api (+service), the three worker queues, scheduler, suspended prisma-migrate CronJob, ingress, and HPA. Full contents in the **Dockerfiles & Helm Chart** artifact — reproduced here in condensed form; use that artifact as the source of truth for the templates.

`infra/helm/Chart.yaml`

```
yaml

apiVersion: v2
name: answerspot
description: AI answer-monitoring platform for local service businesses
type: application
version: 0.1.0
appVersion: "0.1.0"
```

`infra/helm/values.yaml` (key structure)

```
yaml

image: { registry: ghcr.io, repository: your-org/answerspot, tag: latest, pullPolicy:
imagePullSecrets: []
existingSecret: answerspot-secrets # DATABASE_URL, REDIS_URL, CELERY_*, JWT_SECRET,
web: { replicas: 2, resources: {...} }
api: { replicas: 2, port: 4000, resources: {...}, hpa: { enabled: true, minReplicas:
workers:
  runner: { replicas: 2, concurrency: 4, resources: {...}, hpa: { enabled: true, minR
  parser: { replicas: 2, concurrency: 4, resources: {...} }
  recs: { replicas: 1, concurrency: 2, resources: {...} }
scheduler: { replicas: 1, resources: {...} } # exactly one beat
ingress: { enabled: true, className: nginx, tls: true, hosts: { web: app.answerspot.e
config: { scanSampleCount: "3", logLevel: info }
```

Template rules that matter:

- `scheduler` pinned to 1 replica, `strategy: Recreate` — never two beats.
- Each worker deployment overrides `command` with its `-Q <queue>`.
- API probes hit `/api/v1/health` (liveness) and `/api/v1/health/ready` (readiness).
- `prisma-migrate` CronJob is **suspended**; CI creates a Job from it per deploy.
- Secrets come from `existingSecret` (Sealed Secrets/Vault), never values files.

- **Align image naming** between the `answerspot.image` helper (`repository-service:tag`) and the CI push format — resolve before first deploy.

| Copy the complete templates from the Dockerfiles & Helm Chart artifact.

11. CI/CD — GitHub Actions

Three workflows. Full YAML in the **GitHub Actions CI/CD** artifact.

- `.github/workflows/ci.yml` (on PR): node lint/typecheck/test/build against a Postgres service + Prisma migrate; python ruff/mypy/pytest; Trivy fs scan (fails on CRITICAL/HIGH); docker build matrix (no push).
- `.github/workflows/deploy.yml` (on main): build + push `{web,api,worker}` images to GHCR (tags `sha-<sha>` and `staging`) → `helm upgrade` to `answerspot-staging` → migration Job from the CronJob → smoke test `/health`.
- `.github/workflows/release.yml` (on tag `v*`): retag `:staging` → `<tag>` + `:prod` → **manual-approval** `production` environment → migration Job → `helm upgrade` to `answerspot-prod` → post-deploy health check → auto `helm rollback` on failure.

Repo config required: secrets `KUBECONFIG_STAGING`, `KUBECONFIG_PROD` (base64); variables `STAGING_API_URL`, `PROD_API_URL`; environments `staging` (auto) and `production` (required reviewers).

12. ADRs — decisions to resolve

Create at `docs/ADR/`. All start **Proposed**; team fills the decision and flips to **Accepted**. Full write-ups in the **Architecture Decision Records** artifact.

#	Decision	Recommendation
0001	Auth provider (Auth.js vs Clerk)	Auth.js for portability unless speed is overriding
0002	Managed vs self-hosted Postgres/Redis	Managed per cloud; app layer stays agnostic
0003	Celery vs RQ	Celery (native beat + routing fits the pipeline)
0004	AI Overview capture	BLOCKED pending legal ; SERP API if cleared; not in M0-M4
0005	Visibility Score weighting	Fixed documented weights; store weights per snapshot

13. Handoff checklist

- ☐ Create repo; add all files at the paths above
 - ☐ Provision container registry + K8s cluster (staging + prod)
 - ☐ Secrets management (Sealed Secrets/Vault) → populate `answerspot-secrets`
 - ☐ Managed Postgres + Redis (ADR 0002) or Helm equivalents
 - ☐ GitHub Actions: secrets, variables, `staging`/`production` environments (prod = required reviewers)
 - ☐ Align Helm image naming with CI push format
 - ☐ Implement `/health` (done) and wire probes (done)
 - ☐ External AI API keys + budget alerts
 - ☐ Observability: Prometheus/Grafana + Sentry (`SENTRY_DSN`), OTEL endpoint)
 - ☐ Legal review for AI Overview capture (ADR 0004) before M-past-4
 - ☐ Resolve ADRs 0001-0005; flip to Accepted
-

14. Build milestones

Phase	Deliverable	Key <code>TODO</code> markers
M0	Repo scaffold, compose, CI, schema + migrations	—
M1	Auth + onboarding + auto-query generation	<code>TODO(M1)</code> businesses.service
M2	Query Runner (Perplexity first) + scan storage	<code>TODO(M2)</code> runner, scans.service, scheduler, perplexity_adapter
M3	Response Parser + Visibility Score	<code>TODO(M3)</code> parser
M4	Dashboard: Home, Answer Explorer, trend	(web app)
M5	Competitor Radar + Recommendation Engine + artifacts	<code>TODO(M5)</code> recommender, businesses.service
M6	Multi-platform, alerts, Stripe billing	(adapters, billing)
M7	White-label reports, HPA/observability hardening	(reports)

Fundable MVP = M0-M4 — the "oh wow" loop. M5+ is monetization and expansion.

End of master build document. Companion artifacts (full Helm templates, full CI/CD YAML, full ADR write-ups) remain the source of truth for their sections.